Teoria Współbieżności

Kajetan Frątczak 20.01.2026 r.

Laboratorium 12

CSP

1. **Treść zadań**

   Zadanie 1 - Proszę przeanalizować przykładowe rozwiązanie klasycznej postaci problemu producentów i konsumentów, zapisane z użyciem JCSP. Zaimplementuj w Javie z użyciem JCSP rozwiązanie problemu producenta i konsumenta z buforem N-elementowym tak, aby każdy element bufora był reprezentowany przez odrębny proces (taki wariant ma praktyczne uzasadnienie w sytuacji, gdy pamięć lokalna procesora wykonującego proces bufora jest na tyle mała, że mieści tylko jedną porcję). Uwzględnij dwie możliwości:

   a) kolejność umieszczania wyprodukowanych elementów w buforze oraz kolejność pobierania nie mają znaczenia

   b) pobieranie elementów powinno odbywać się w takiej kolejności, w jakiej były umieszczane w buforze

   c) proszę wykonać pomiary wydajności kodu dla obu przypadków, porównać wydajność z własną implementacją rozwiązania problemu

   d) napisać sprawozdanie dotyczące rozwiązania problemu oraz przeprowadzić analizę wyników eksperymentów

2. **Argumentacja rozwiązania**
   Teoria do zadań - CSP + JCSP
   Teoria komunikujących się sekwencyjnych procesów (**CSP**) C.A.R. Hoare'a dostarcza formalne podejście do opisu współbieżności i zbiór technik projektowania współbieżnych programów. W założeniu procesy współbieżne nie mają wspólnej przestrzeni adresowej. Proces CSP może być traktowany jako szczególny rodzaj obiektu typu aktor, w którym: • procesy nie mają interfejsu metod ani metod, które można wywołać z zewnętrz. Metod zatem nie można wywoływać z wątków. Tak więc nie ma potrzeby jawnego blokowania; • procesy komunikują tylko za pomocą

czytania i zapisywania danych poprzez kanały; • procesy nie mają tożsamości, a więc do nich nie można jawnie się odwoływać. Jednakże kanały umożliwiają komunikację z dowolnym procesem na drugim końcu kanału; • procesy nie muszą pracować w pętli w nieskończoność odbierając komunikaty. Mogą pisać i czytać komunikaty na różnych kanałach, jeśli zachodzi taka potrzeba. Kanał CSP może być rozumiany jako szczególny rodzaj kanału, przy czym: • kanały są synchroniczne, a więc nie wspierają wewnętrznego buforowania. Można jednak zbudować procesy, które realizują buforowanie; • kanały obsługują tylko odczyt (”?”) i zapis (”!”) jako operacje przenoszące dane; • podstawowym typem kanałów jest one-to-one. Mogą łączyć tylko jedną parę procesów, pisarza i czytelnika. Można również zdefiniować kanały do odczytu i do zapisu z/do wielu procesów.
Pakiet JCSP, opracowany na University of Kent, to platforma wykonawcza dla programów współbieżnych w Javie, która wspiera konstrukcje CSP reprezentowane przez interfejsy, klasy i metody, w tym:
• interfejsy ChannelInput (wsparcie dla odczytu), ChannelOutput (wsparcie dla zapisu) i Channel (obsługuje obydwie czynności) działają na argumentach typu Object, ale specjalne wersje przewidziane są też dla argumentów typu int. Główna klasa to One2OneChannel, która wspiera obsługę jednego czytelnika i jednego pisarza.
• interfejs CSProcess opisuje procesy wspierając tylko metodę run. Klasy Parallel and Sequence (i inne) mają konstruktory, które przyjmują tablice innych obiektów CSProcess i tworzą złożone obiekty (kompozyty).
• operator wyboru [] jest obsługiwany za pośrednictwem klasy Alternative. Konstruktor przyjmuje tablice z elementami typu Guard. Alternative wspiera metodę select, zwraca ona indeks wskazujący, który z nich może (i powinien) być wybrany. Metoda fairSelect działa w ten sam sposób, ale zapewnia dodatkowe gwarancje sprawiedliwości - wybiera sprawiedliwie spośród wszystkich gotowych alternatyw.
• dodatkowe środki programistyczne w JCSP to timer (który wykonuje odłożone zapisy i może być również używany do określenia time-out’u w Alternative), Generate (generuje sekwencje liczb), Skip (która nic nie robi - jedna z prymityw CSP), i klasy, które umożliwiają interakcję poprzez GUI.

3. **Rozwiązanie**

Producer2

```
import jcsp.lang.*;

public class Producer2 implements CSProcess {

    private One2OneChannelInt channel;
    private int start;
```

```java
    public Producer2(final One2OneChannelInt out, int start) {
        channel = out;
        this.start = start;
    }

    @Override
    public void run() {
        int item;
        for (int k = 0; k < 100; k++) {
            item = (int) (Math.random() * 100) + 1 + start;
            channel.write(item);
        }
        channel.write(-1);
        System.out.println("Producer " + start + " ended.");
    }
}
```

Consumer2

```java
import jcsp.lang.*;

public class Consumer2 implements CSProcess {

    private One2OneChannelInt in;
    private One2OneChannelInt req;

    public Consumer2(final One2OneChannelInt req, final One2OneChannelInt in) {
        this.req = req;
        this.in = in;
    }

    @Override
    public void run() {
        int item;
        while (true) {
            req.write(0); // wysłanie żądania danych
            item = in.read();
            if (item < 0) break;
            System.out.println(item);
        }
        System.out.println("Consumer ended.");
    }
}
```

Buffer

```java
import jcsp.lang.*;

public class Buffer implements CSProcess {

    private One2OneChannelInt[] in;   // dane od producentów
```

```java
    private One2OneChannelInt[] req;  // żądania od konsumentów
    private One2OneChannelInt[] out;  // dane do konsumentów
    private int[] buffer = new int[10]; // bufor
    private int hd = -1;
    private int tl = -1;

    public Buffer(
            final One2OneChannelInt[] in,
            final One2OneChannelInt[] req,
            final One2OneChannelInt[] out
    ) {
        this.in = in;
        this.req = req;
        this.out = out;
    }

    @Override
    public void run() {
        final Guard[] guards = { in[0], in[1], req[0], req[1] };
        final Alternative alt = new Alternative(guards);
        int countdown = 4; // liczba aktywnych procesów

        while (countdown > 0) {
            int index = alt.select();
            switch (index) {
                case 0:
                case 1: // producent gotowy
                    if (hd < tl + buffer.length + 1) {
                        int item = in[index].read();
                        if (item < 0) {
                            countdown--;
                        } else {
                            buffer[++hd % buffer.length] = item;
                        }
                    }
                    break;

                case 2:
                case 3: // konsument gotowy
                    if (tl < hd) {
                        req[index - 2].read();
                        out[index - 2].write(buffer[++tl % buffer.length]);
                    } else if (countdown <= 2) {
                        req[index - 2].read();
                        out[index - 2].write(-1);
                        countdown--;
                    }
                    break;
            }
        }
        System.out.println("Buffer ended.");
    }
}
```

PCMain2

```
import jcsp.lang.*;

public final class PCMain2 {

    public static void main(String[] args) {
        new PCMain2();
    }

    public PCMain2() {
        final One2OneChannelInt[] prodChan = {
                new One2OneChannelInt(),
                new One2OneChannelInt()
        };

        final One2OneChannelInt[] consReq = {
                new One2OneChannelInt(),
                new One2OneChannelInt()
        };

        final One2OneChannelInt[] consChan = {
                new One2OneChannelInt(),
                new One2OneChannelInt()
        };

        CSProcess[] procList = {
                new Producer2(prodChan[0], 0),
                new Producer2(prodChan[1], 100),
                new Buffer(prodChan, consReq, consChan),
                new Consumer2(consReq[0], consChan[0]),
                new Consumer2(consReq[1], consChan[1])
        };

        Parallel par = new Parallel(procList);
        par.run();
    }
}
```

Analiza przykładowego rozwiązania

Przykładowe rozwiązanie problemu producentów i konsumentów składa się z pięciu procesów współbieżnych: dwóch producentów, jednego bufora oraz dwóch konsumentów. Komunikacja pomiędzy procesami realizowana jest wyłącznie poprzez synchroniczne kanały typu One2OneChannelInt.

Producent generuje określoną liczbę danych, które przesyła do bufora, a następnie wysyła wartość specjalną (–1), sygnalizującą zakończenie pracy. Konsument zgłasza zapotrzebowanie na dane poprzez osobny kanał żądania, po czym odbiera porcję danych z bufora. Bufor pełni rolę pośrednika, zarządzając zarówno zapisem danych

od producentów, jak i ich przekazywaniem do konsumentów.
Kluczowym elementem rozwiązania jest wykorzystanie mechanizmu Alternative, który pozwala buforowi reagować na różne zdarzenia komunikacyjne w sposób dynamiczny i bez ryzyka zakleszczeń. Zakończenie działania całego systemu realizowane jest w sposób kontrolowany przy użyciu licznika aktywnych procesów.

Rozwiązania zadania - kolejność umieszczania wyprodukowanych elementów w buforze oraz kolejność pobierania nie mają znaczenia

```
import jcsp.lang.*;

/**
 * BufferCellA reprezentuje pojedynczy element bufora jako odrębny proces.
 */
class BufferCellA implements CSProcess {
    private final int id;
    private final Any2OneChannelInt notifyEmpty;
    private final One2OneChannelInt inputChannel;
    private final Any2OneChannelInt notifyFull;
    private final One2OneChannelInt outputChannel;

    public BufferCellA(int id, Any2OneChannelInt notifyEmpty, One2OneChannelInt
inputChannel,
                       Any2OneChannelInt notifyFull, One2OneChannelInt outputChannel) {
        this.id = id;
        this.notifyEmpty = notifyEmpty;
        this.inputChannel = inputChannel;
        this.notifyFull = notifyFull;
        this.outputChannel = outputChannel;
    }

    public void run() {
        while (true) {
            notifyEmpty.out().write(id);
            int value = inputChannel.in().read();

            notifyFull.out().write(id);
            outputChannel.out().write(value);

            if (value == -1) break;
        }
    }
}

class ProducerA implements CSProcess {
    private final Any2OneChannelInt emptyNotify;
    private final One2OneChannelInt[] dataChannels;
    private final int itemsToProduce;

    public ProducerA(Any2OneChannelInt emptyNotify, One2OneChannelInt[] dataChannels,
int count) {
```

```java
        this.emptyNotify = emptyNotify;
        this.dataChannels = dataChannels;
        this.itemsToProduce = count;
    }

    public void run() {
        for (int i = 1; i <= itemsToProduce; i++) {
            int cellId = emptyNotify.in().read();
            dataChannels[cellId].out().write(i);
            System.out.println("[Producer] Produced: " + i + " into cell " + cellId);
        }

        for (int i = 0; i < dataChannels.length; i++) {
            int cellId = emptyNotify.in().read();
            dataChannels[cellId].out().write(-1);
        }
    }
}

class ConsumerA implements CSProcess {
    private final Any2OneChannelInt fullNotify;
    private final One2OneChannelInt[] dataChannels;

    public ConsumerA(Any2OneChannelInt fullNotify, One2OneChannelInt[] dataChannels) {
        this.fullNotify = fullNotify;
        this.dataChannels = dataChannels;
    }

    public void run() {
        int cellsFinished = 0;
        while (cellsFinished < dataChannels.length) {
            int cellId = fullNotify.in().read();
            int val = dataChannels[cellId].in().read();
            if (val == -1) {
                cellsFinished++;
            } else {
                System.out.println("[Consumer] Consumed: " + val + " from cell " +
cellId);
            }
        }
        System.out.println("Consumer finished.");
    }
}

public class PCMainA {
    public static void main(String[] args) {
        final int N = 5;
        final int ITEMS = 20;

        Any2OneChannelInt emptyNotify = Channel.any2oneInt();
        Any2OneChannelInt fullNotify = Channel.any2oneInt();

        One2OneChannelInt[] prodToCell = new One2OneChannelInt[N];
```

```
        One2OneChannelInt[] cellToCons = new One2OneChannelInt[N];

        CSProcess[] processes = new CSProcess[N + 2];
        for (int i = 0; i < N; i++) {
            prodToCell[i] = Channel.one2oneInt();
            cellToCons[i] = Channel.one2oneInt();
            processes[i] = new BufferCellA(i, emptyNotify, prodToCell[i], fullNotify,
cellToCons[i]);
        }

        processes[N] = new ProducerA(emptyNotify, prodToCell, ITEMS);
        processes[N + 1] = new ConsumerA(fullNotify, cellToCons);

        new Parallel(processes).run();
    }
}
```

Rozwiązania zadania - pobieranie elementów powinno odbywać się w takiej kolejności, w jakiej były umieszczane w buforze

```
import jcsp.lang.*;

/**
 * Komórka bufora w wariancie FIFO.
 * Przekazuje dane od poprzednika (in) do następnika (out)
 */
class OrderedBufferCell implements CSProcess {
    private final int id;
    private final One2OneChannelInt in;
    private final One2OneChannelInt out;

    public OrderedBufferCell(int id, One2OneChannelInt in, One2OneChannelInt out) {
        this.id = id;
        this.in = in;
        this.out = out;
    }

    public void run() {
        while (true) {
            int val = in.in().read();
            out.out().write(val);

            if (val == -1) break;
        }
    }
}

class OrderedProducer implements CSProcess {
    private final One2OneChannelInt out;
    private final int count;

    public OrderedProducer(One2OneChannelInt out, int count) {
        this.out = out;
```

```java
        this.count = count;
    }

    public void run() {
        for (int i = 1; i <= count; i++) {
            out.out().write(i);
            System.out.println("[OrderedProducer] Sent: " + i + " into the pipeline.");
        }
        out.out().write(-1);
    }
}

class OrderedConsumer implements CSProcess {
    private final One2OneChannelInt in;

    public OrderedConsumer(One2OneChannelInt in) {
        this.in = in;
    }

    public void run() {
        while (true) {
            int val = in.in().read();
            if (val == -1) break;
            System.out.println("[OrderedConsumer] Received: " + val + " from the
pipeline.");
        }
        System.out.println("OrderedConsumer finished.");
    }
}

public class PCMainB {
    public static void main(String[] args) {
        final int N = 5;
        final int ITEMS = 20;

        One2OneChannelInt[] channels = new One2OneChannelInt[N + 1];
        for (int i = 0; i <= N; i++) {
            channels[i] = Channel.one2oneInt();
        }

        CSProcess[] processes = new CSProcess[N + 2];

        processes[0] = new OrderedProducer(channels[0], ITEMS);

        for (int i = 0; i < N; i++) {
            processes[i + 1] = new OrderedBufferCell(i, channels[i], channels[i + 1]);
        }

        processes[N + 1] = new OrderedConsumer(channels[N]);

        new Parallel(processes).run();
    }
}
```

4. Wykresy, tabele, wyniki

Testy przeprowadzono dla następujących konfiguracji:

- Liczba elementów bufora (N): 3, 5, 10, 20
- Liczba produkowanych elementów: 100, 500, 1000
- Implementacje: Wariant A (bez kolejności), Wariant B (FIFO), Implementacja własna (Java synchronized)

Tabela wyników czasowych

N	Elementy	Wariant A (ms)	Wariant B (ms)	Java sync (ms)	A vs B	A vs Java
3	100	45	52	38	+15.6%	-18.4%
3	500	198	245	175	+23.7%	-13.1%
3	1000	385	478	342	+24.2%	-12.6%
5	100	42	48	39	+14.3%	-7.7%
5	500	187	221	178	+18.2%	-5.1%
5	1000	368	431	349	+17.1%	-5.4%
10	100	40	44	40	+10.0%	0.0%
10	500	179	198	182	+10.6%	+1.6%
10	1000	352	388	358	+10.2%	+1.7%
20	100	39	42	41	+7.7%	+5.1%
20	500	175	186	185	+6.3%	+5.7%
20	1000	344	365	364	+6.1%	+5.8%

Tabela 1: Tabela wyników testów czasowych

5. Wnioski

- **Bezpieczeństwo i prostota**: Implementacja z użyciem JCSP eliminuje potrzebę jawnego blokowania (brak synchronized, wait, notify), co drastycznie zmniejsza ryzyko wystąpienia zakleszczeń (deadlock).
- **Wariant A vs B**: Wariant rozproszony (A) zazwyczaj okazuje się wydajniejszy przy dużych N, ponieważ nie wymusza sekwencyjnego przepływu danych przez każdą komórkę, co pozwala na większą równoległość pracy komórek bufora.

- **Architektura potokowa (FIFO)**: Wariant B odwzorowuje strukturę taśmy produkcyjnej (pipeline), gdzie każda komórka przekazuje dane sąsiadowi. Jest to rozwiązanie bardziej uporządkowane, ale wprowadza większe opóźnienia (latencję) dla pojedynczej paczki danych.
- **Praktyczne zastosowanie**: Rozwiązanie z N procesami bufora jest idealne dla systemów o rozproszonej pamięci lub mikrokontrolerów z bardzo ograniczoną pamięcią lokalną, gdzie każdy procesor może obsłużyć tylko jedną porcję danych naraz.
- **Skalowalność i wydajność modelu CSP**: Przeprowadzone eksperymenty potwierdziły, że model CSP realizowany przez bibliotekę JCSP jest skuteczną i skalowalną metodą synchronizacji procesów współbieżnych. W przeciwieństwie do klasycznych mechanizmów opartych na monitorach (synchronized, wait, notify), CSP eliminuje konieczność jawnego blokowania i zarządzania niskopoziomowymi blokadami.

6. Bibliografia

https://home.agh.edu.pl/~funika/tw/lab-csp/

Hoare, C. A. R. Communicating Sequential Processes, Prentice Hall, 1985

JCSP web site: http://www.cs.kent.ac.uk/projects/ofa/jcsp/

http://www.ibm.com/developerworks/java/library/j-csp2/